

Reto DataQuantum

Resumen visual del Nivel Básico

Asistente de terminal en Python conectado a un LLM, desarrollado paso a paso, probado y documentado.

Stack técnico

- Python 3.13.15
- Groq + openai/gpt-oss-120b
- groq==1.6.0
- python-dotenv==1.2.3
- Coste objetivo: 0 €

Seguridad

- .env para la clave real
- .env.example sin credenciales
- .gitignore protege .env y .venv
- La API key nunca se versiona

Estado del repo

main = origin/main

Historial limpio y progresivo

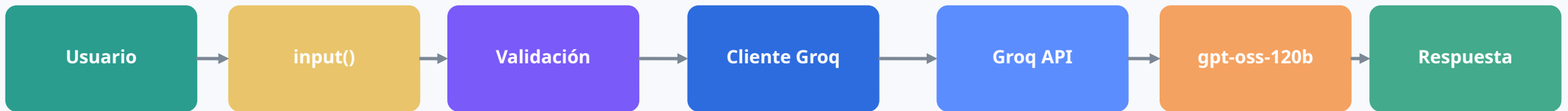
100%

**Nivel Básico
completado**



1. Arquitectura del asistente

Flujo completo de una pregunta desde la terminal hasta el modelo y vuelta.



Entrada vacía

strip() elimina espacios. Si queda vacío:

continue → volver a pedir

Comando salir

casefold() reconoce salir / SALIR / Salir

break → finalizar bucle

Error de API

try/except evita que un fallo cierre el asistente

continue → seguir usando

Principio clave: el cliente de Groq se crea una sola vez antes del while.

2. Estructura del proyecto y evolución en Git

El repositorio muestra cómo se construyó el nivel por etapas, no como un bloque único.

Estructura del repositorio

```
reto-agentes-ia-dataquantum /  
├── src/  
│   ├── prueba_conexion.py  
│   └── nivel_basico.py  
├── docs/  
│   └── diario_decisiones.md  
├── .env (ignorado)  
├── .env.example  
├── .gitignore  
├── requirements.txt  
└── README.md
```

.env y .venv nunca se suben

Línea temporal de commits



Resultado: cada fase queda trazable, revisable y fácil de explicar.

3. Qué significa “sin memoria” en el Nivel Básico

Cada llamada contiene solo la pregunta actual. Esto es deliberado y marca la frontera con el Nivel Intermedio.

Nivel Básico: peticiones independientes

Petición 1

```
{ role: "user", content: "Me llamo JP" }
```

Petición 2

```
{ role: "user", content: "¿Cómo me llamo?" }
```

La segunda llamada NO recibe la primera.

messages se crea de nuevo

Nivel Intermedio: memoria conversacional

m essages = [

system Eres un asistente útil

user Me llamo JP

assistant Encantado, JP

user ¿Cómo me llamo?

]

Aquí sí se envía el contexto acumulado.

4. Pruebas realizadas

El Nivel Básico se cerró solo después de validar el flujo con casos normales y de borde.



Entrada vacía

Muestra "Debes escribir una pregunta.", no llama a la API y vuelve al inicio.



SALIR

Reconoce mayúsculas/minúsculas y finaliza sin llamar a Groq.



Varias preguntas

Roma. / 15 / Hasta luego. en una sola sesión, con exactamente dos llamadas.



Sin memoria

Cada petición crea una lista messages nueva con solo la pregunta actual.

Resultado final: no se detectaron errores funcionales en las pruebas de cierre.

5. Decisiones técnicas y aprendizaje

Qué elegimos, por qué lo elegimos y qué incidencia real apareció durante el desarrollo.

Decisiones que quedan en el diario

- Python 3.13.15 para compatibilidad y estabilidad.
- Groq para mantener el reto en coste 0 €.
- openai/gpt-oss-120b como modelo inicial.
- python-dotenv para cargar GROQ_API_KEY desde .env.
- Sin LangChain/LangGraph todavía: primero entender la API directa.
- Identificadores internos en inglés; interfaz en español.
- Construcción incremental: conexión → una pregunta → bucle completo.

Incidencia real documentada

Problema	La captura de terminal mostró “conexión correcta”.
Causa	Codificación de la terminal/captura, no fallo de Groq.
Resultado	La respuesta real era “conexión correcta” y la API funcionaba.

Criterio para documentar errores

- Problema
 - Causa
 - Solución
 - Resultado
- Solo se documentan incidencias relacionadas con el desarrollo del asistente; no se rellenan errores inventados ni problemas externos irrelevantes.

6. Estado actual y siguiente paso

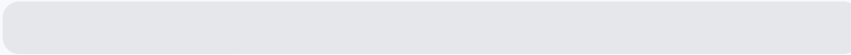
El Nivel Básico queda congelado como evidencia del aprendizaje; el siguiente archivo será nivel_intermedio.py.

Progreso del reto

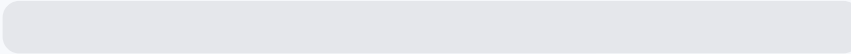
Básico



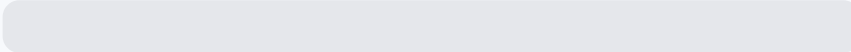
Intermedio



Avanzado



Experto



Idea clave para recordar

**Básico = cada pregunta independiente.
Intermedio = contexto acumulado.**

10

Nivel Intermedio: objetivos

- Crear src/nivel_intermedio.py sin modificar el Nivel Básico.
- Mantener una lista messages durante toda la conversación.
- Usar roles system / user / assistant.
- Comprobar que el modelo recuerda contexto previo.
- Guardar la conversación en un archivo JSON con fecha.
- Experimentar con distintos system prompts y documentar diferencias.